DERIVATIVE PRICING

MODULE 7 | LESSON 1

STOCHASTIC VOLATILITY: HESTON

Reading Time	50 minutes
Prior Knowledge	Black-Scholes, Local-volatility, Stochastic-volatility
Keywords	Stochastic volatility, Heston model, Skewness, Kurtosis

---

*In Lesson 1 of Module 7, we start dealing with **stochastic volatility models**. Specifically, we will implement the **Monte-Carlo simulation** of the **Heston (1993)** stochastic volatility model.*

As usual, let's start by importing the necessary libraries:

In [10]:
```
import matplotlib.pyplot as plt
import numpy as np
import scipy.stats as ss
```

1. Simulating the Heston Model in Python

As you have already seen in the slides, we can simulate stock price paths under Heston model dynamics with the following two equations: $$ \begin{equation*} S_t = S_{t-1} e^{\left(r - \frac{\nu_t}{2} \right) dt + \sqrt{\nu_t} dZ_1} \end{equation*} $$ $$ \begin{equation*} \nu_t = \nu_{t-1} + \kappa \left(\theta - \nu_{t-1} \right) dt + \sigma \sqrt{\nu_{t-1}}dZ_2 \end{equation*} $$
Importantly, these equations require some parameters, whose functional form is currently unknown to us. At this step, we will just give some values to these parameters to generate sample paths. In the next lesson, we will look at some semi-analytical ways (e.g., Fourier transform methods) to obtain a functional form for these.

We will start by coding the **stochastic volatility function**, ν_t:

Notice how we calculate ν_t as the maximum between the ν_t expression above and 0. This is to ensure that we do not get a negative number here because of the random process. Obviously, it does not make any sense whatsoever to have negative variance!

In [11]:
```
def SDE_vol(v0, kappa, theta, sigma, T, M, I, rand, row, cho_matrix):
    np.random.seed(2)
    dt = T / M  # T = maturity, M = number of time steps
    v = np.zeros((M + 1, I), dtype=float)
    v[0] = v0
```

```
    sdt = np.sqrt(dt)  # Sqrt of dt
    for t in range(1, M + 1):
        ran = np.dot(cho_matrix, rand[:, t])
        v[t] = np.maximum(0, v[t - 1] + kappa * (theta - v[t - 1]) * dt + np.sqr
    return v
```

Next, let's implement the classic **stochastic equation** for the underlying asset price evolution:

In [12]:
```
def Heston_paths(S0, r, v, row, cho_matrix):
    np.random.seed(2)
    S = np.zeros((M+1, I), dtype=float)
    S[0] = S0
    sdt = np.sqrt(dt)
    for t in range(1, M+1, 1):
        ran = np.dot(cho_matrix, rand[:,t])
        S[t] = S[t - 1] * np.exp((r - 0.5 * v[t-1]) * dt + np.sqrt(v[t-1]) * ran

    return S
```

The last function we will define, just for simplifying the tasks, will be a **random number generator** following a standard normal:

In [13]:
```
def random_number_gen(M, I):
    np.random.seed(2)
    rand = np.random.standard_normal((2, M+1, I))
    return rand
```

Now that we have defined the main functions to use in the process, let's implement the Heston model under some parameters (assume these are given for now):

In [14]:
```
v0 = 0.05
kappa_v = 2
sigma_v = 0.3
theta_v = 0.04
rho = -0.9

S0 = 100  # Current underlying asset price
r = 0.05  # Risk-free rate
M0 = 5   # Number of time steps in a year
T = 1  # Number of years
M = int(M0*T) # Total time steps
I = 1  # Nomber of simulations
dt = T/M  # Length of time step
```

Finally, let's work on the random number generator and the covariance matrix (using Cholesky decomposition to account for correlation between dZ_1 and dZ_2):

Finally, let's work on the random number generator and the covariance matrix (using Cholesky decomposition to account for correlation between dZ_1 and dZ_2):

In [15]:
```
# Generating random numbers from standard normal
np.random.seed(2)
rand = random_number_gen(M, I)


# Covariance Matrix
```

```
covariance_matrix = np.zeros((2, 2))
covariance_matrix[0] = [1.0, rho]
covariance_matrix[1] = [rho, 1.0]
cho_matrix = np.linalg.cholesky(covariance_matrix)
```

Now we have all the ingredients to generate the paths for both asset price and its volatility:

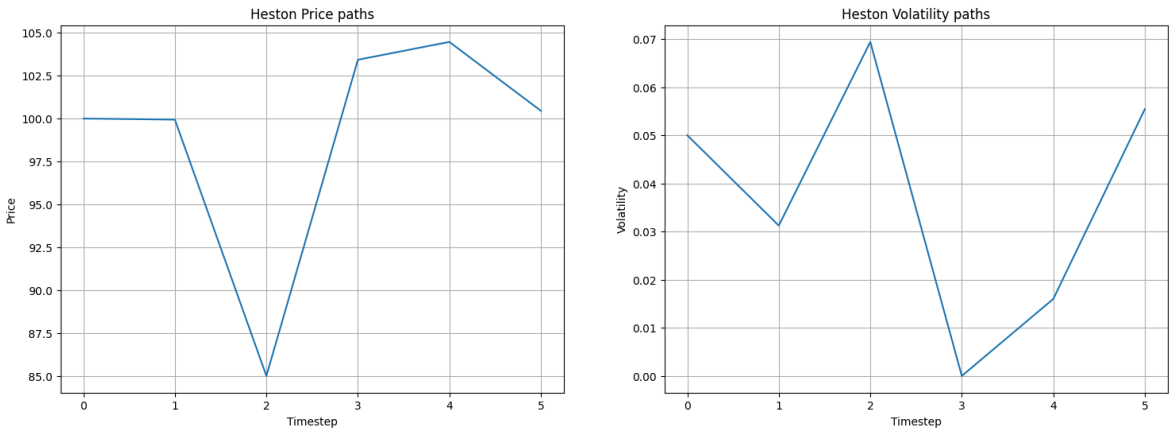
In [16]:
```
# Volatility process paths
V = SDE_vol(v0, kappa_v, theta_v, sigma_v, T, M, I, rand, 1, cho_matrix)

# Underlying price process paths
S = Heston_paths(S0, r, V, 0, cho_matrix)
```

In [17]:
```
S
```

Out[17]:
```
array([[100.        ],
       [ 99.93735136],
       [ 84.99390527],
       [103.42812163],
       [104.46759154],
       [100.45003833]])
```

Let's visualize some of the paths for both the underlying price and the volatility:

In [18]:
```
def plot_paths(n):
    fig = plt.figure(figsize=(18, 6))
    ax1 = fig.add_subplot(121)
    ax2 = fig.add_subplot(122)

    ax1.plot(range(len(S)), S[:, :n])
    ax1.grid()
    ax1.set_title("Heston Price paths")
    ax1.set_ylabel("Price")
    ax1.set_xlabel("Timestep")

    ax2.plot(range(len(V)), V[:, :n])
    ax2.grid()
    ax2.set_title("Heston Volatility paths")
    ax2.set_ylabel("Volatility")
    ax2.set_xlabel("Timestep")


plot_paths(100)
```



2. The Statistical Distribution Produced by Heston

One important feature of the Heston model is that it produces a distribution of returns that has **heavier tails and kurtosis** than a normal distribution, just as we observe in practice.

In the following code snippet, we will check the Heston-produced distribution on two fronts:

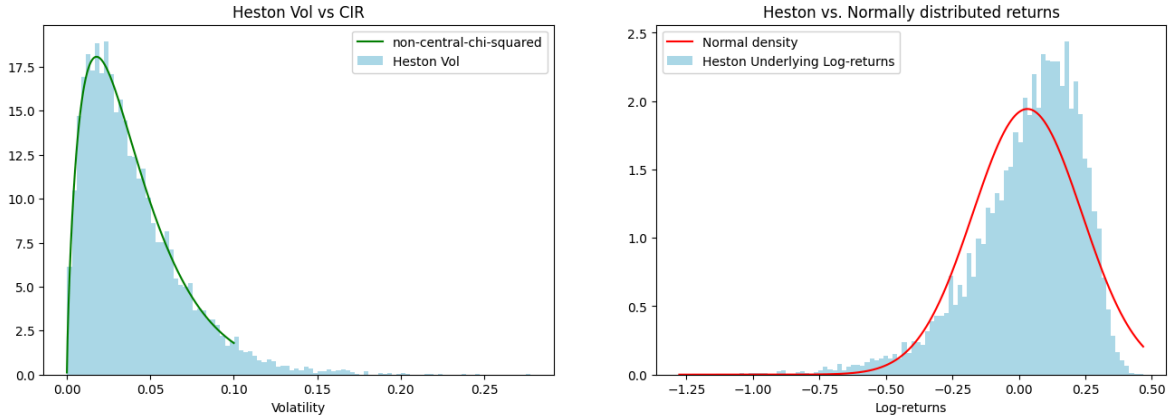
1. Whether **underlying returns** resemble a Normal distribution, as assumed in BS and GBM process.

2. How **volatility** fits a mean-reverting process such as CIR or Vasicek.

In [20]:
```
# Obtaining degrees of freedom (do not worry a lot about this now)
c = 2 * kappa_v / ((1 - np.exp(-kappa_v * T)) * sigma_v**2)
df = 4 * kappa_v * theta_v / sigma_v**2
nc = 2 * c * v0 * np.exp(-kappa_v * T)


# Calculating returns and lengths of axis
log_R = np.log(S[-1, :] / S0)
x = np.linspace(log_R.min(), log_R.max(), 500)
y = np.linspace(0.00001, 0.1, 500)


# Plotting the different distributions
fig = plt.figure(figsize=(16, 5))
ax1 = fig.add_subplot(121)
ax2 = fig.add_subplot(122)

# Heston stochastic vol follows a CIR/Vasicek process
ax1.plot(
    y,
    ss.ncx2.pdf(y, df, nc, scale=1 / (2 * c)),
    color="green",
    label="non-central-chi-squared",
)
ax1.hist(V[-1, :], density=True, bins=100, facecolor="LightBlue", label="Heston
ax1.legend()
ax1.set_title("Heston Vol vs CIR")
ax1.set_xlabel("Volatility")

# Heston underlying returns do not follow a normal distribution
ax2.plot(
    x,
    ss.norm.pdf(x, log_R.mean(), log_R.std(ddof=0)),
    color="r",
    label="Normal density",
)
ax2.hist(
    log_R,
    density=True,
    bins=100,
```

```
    facecolor="LightBlue",
    label="Heston Underlying Log-returns",
)
ax2.legend()
ax2.set_title("Heston vs. Normally distributed returns")
ax2.set_xlabel("Log-returns")
```

Out[20]: Text(0.5, 0, 'Log-returns')



In the previous graphs, you can observe the fit (and no-fit) of the different distributions produced to known distributions assumed in other models.

3. Conclusion

Well done! Now you know a lot more about how to implement the Heston model in Python. Arguably, one thing we have not yet covered explicitly is pricing in the Heston framework. This is exactly what we will do in the next lesson.

Copyright 2025 WorldQuant University. This content is licensed solely for personal use. Redistribution or publication of this material is strictly prohibited.

DERIVATIVE PRICING

MODULE 7 | LESSON 2

OPTION PRICING UNDER HESTON (MONTE CARLO)

Reading Time	70 minutes
Prior Knowledge	Black-Scholes, Heston, Stochastic volatility
Keywords	Monte Carlo, Heston, Black-Scholes

---

*In Lesson 2 of Module 7, we will **implement Monte-Carlo pricing** of an option **under Heston model dynamics**.*

As usual, let's start by importing the necessary libraries:

In [1]:
```
import matplotlib.pyplot as plt
import numpy as np
import scipy.stats as ss
```

1. Option Pricing under Heston Using Monte-Carlo Methods

As you can expect, we will be first replicating the code used in the last lesson on how to simulate underlying stock prices under the following Heston dynamics: $$ \begin{equation*} S_t = S_{t-1} e^{\left(r - \frac{\nu_t}{2} \right) dt + \sigma \sqrt{\nu_t} dZ_1} \end{equation*} $$ $$ \begin{equation*} \nu_t = \nu_{t-1} + \kappa \left(\theta - \nu_{t-1} \right) dt + \sigma \sqrt{\nu_{t-1}}dZ_2 \end{equation*} $$
So, let's define and run the different functions we needed to simulate stock prices:

- First, **stochastic volatility**:

In [2]:
```
def SDE_vol(v0, kappa, theta, sigma, T, M, I, rand, row, cho_matrix):
    dt = T / M  # T = maturity, M = number of time steps
    v = np.zeros((M + 1, I), dtype=float)
    v[0] = v0
    sdt = np.sqrt(dt)  # Sqrt of dt
    for t in range(1, M + 1):
        ran = np.dot(cho_matrix, rand[:, t])
        v[t] = np.maximum(0, v[t - 1] + kappa * (theta - v[t - 1]) * dt + np.sqr
    return v
```

- Second, **underlying stock price** path based on stochastic vol:

In [3]:
```
def Heston_paths(S0, r, v, row, cho_matrix):
    S = np.zeros((M+1, I), dtype=float)
    S[0] = S0
    sdt = np.sqrt(dt)
    for t in range(1, M+1, 1):
        ran = np.dot(cho_matrix, rand[:,t])
        S[t] = S[t - 1] * np.exp((r - 0.5 * v[t-1]) * dt + np.sqrt(v[t-1]) * ran

    return S
```

- Third, our function to **generate random numbers** following a standard normal:

In [4]:
```
def random_number_gen(M, I):
    rand = np.random.standard_normal((2, M+1, I))
    return rand
```

We have already discussed in the slides that one of the problems we will encounter when using Heston is how to set up the different parameters needed by the model. We will spend a lot of time unveiling this in the Stochastic Modeling course. For now, let's assume some given parameters, as we did in Lesson 1:

In [5]:
```
v0 = 0.095
kappa_v = 5
sigma_v = 0.35
theta_v = 0.1
rho = 0.25

S0 = 40.53  # Current underlying asset price
r = 0.085  # Risk-free rate
M0 = 500   # Number of time steps in a year
T = 4/12  # Number of years
M = int(M0*T) # Total time steps
I = 20000  # Nomber of simulations
dt = T/M  # Length of time step

np.random.seed(42)
```

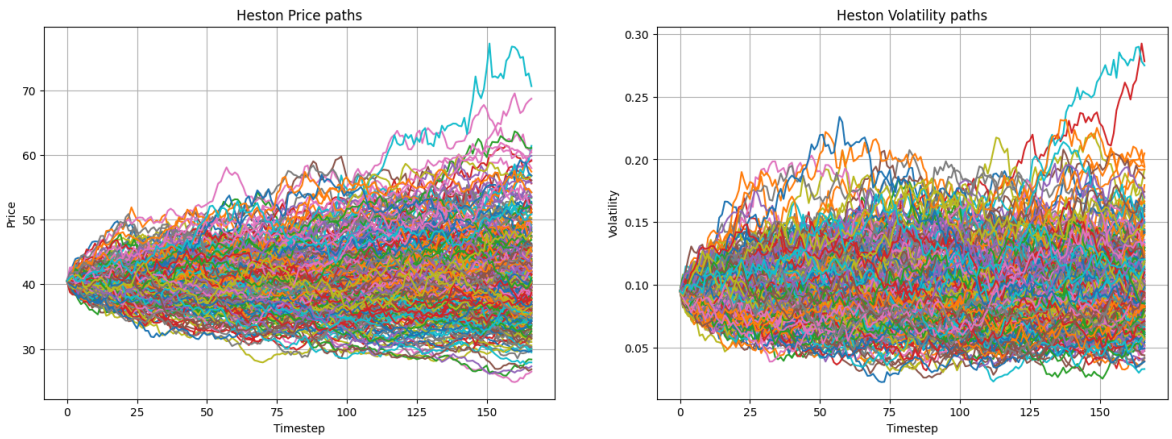
And build our covariance matrix (using Cholesky decomposition to account for correlation between dZ_1 and dZ_2):

In [6]:
```
# Generating random numbers from standard normal
rand = random_number_gen(M, I)


# Covariance Matrix
covariance_matrix = np.zeros((2, 2))
covariance_matrix[0] = [1.0, rho]
covariance_matrix[1] = [rho, 1.0]
cho_matrix = np.linalg.cholesky(covariance_matrix)
```

So, as in the previous lesson, we can now simulate the stock price paths and stochastic volatility:

In [7]:
```
# Volatility process paths
V = SDE_vol(v0, kappa_v, theta_v, sigma_v, T, M, I, rand, 1, cho_matrix)

# Underlying price process paths
S = Heston_paths(S0, r, V, 0, cho_matrix)
```

In [8]:
```
def plot_paths(n):
    fig = plt.figure(figsize=(18, 6))
    ax1 = fig.add_subplot(121)
    ax2 = fig.add_subplot(122)

    ax1.plot(range(len(S)), S[:, :n])
    ax1.grid()
    ax1.set_title("Heston Price paths")
    ax1.set_ylabel("Price")
    ax1.set_xlabel("Timestep")

    ax2.plot(range(len(V)), V[:, :n])
    ax2.grid()
    ax2.set_title("Heston Volatility paths")
    ax2.set_ylabel("Volatility")
    ax2.set_xlabel("Timestep")


plot_paths(300)
```



Now that we have simulated all the stock prices under Heston, let's define another function that calculates the payoffs and current price of a European call option with the following characteristics:

- $T=1$ year
- $K = 90$
- $r=0.05$
- $S_t \sim$ Heston dynamics

In [17]:
```
def heston_call_mc(S, K, r, T, t):
    payoff = np.maximum(0, S[-1, :] - K)

    average = np.mean(payoff)

    return np.exp(-r * (T - t)) * average
```

Note that you can improve the efficiency of the previous function substantially by implementing a better and more organized function that incorporates the generation of stock price and volatility paths. We recommend you work on this by yourself (it should be fairly easy) in order to work on Group Work Projects and other tasks. At this point, we just want to highlight how we apply the spirit of the Monte-Carlo method in simpler frameworks.

So, the price of a European Call option with 1 year maturity and strike $K=90$ (and all the other assumed parameters from before) is...

In [33]:
```
print("European Call Price under Heston: ", heston_call_mc(S, 39, r, T, 0))
```

```
European Call Price under Heston:  4.2765383230635745
```

2. Comparison of Heston Option Price versus Other Methods

Let's now see how the value of the option via Heston model differs from what we would have under the Black-Scholes framework, for example.

- **Black-Scholes closed-form solution**:

In [11]:
```
def bs_call_price(S, r, sigma, t, T, K):
    ttm = T - t

    if ttm < 0:
        return 0.0
    elif ttm == 0.0:
        return np.maximum(S - K, 0.0)

    vol = sigma * np.sqrt(ttm)

    d_minus = np.log(S / K) + (r - 0.5 * sigma**2) * ttm
    d_minus /= vol

    d_plus = d_minus + vol

    res = S * ss.norm.cdf(d_plus)
    res -= K * np.exp(-r * ttm) * ss.norm.cdf(d_minus)

    return res
```

In [12]:
```
print("European Call Price under BS: ", bs_call_price(100, 0.05, sigma_v, 0, 1,
```

```
European Call Price under BS:  19.697442086839736
```

- **Black-Scholes Monte-Carlo price:**

In [13]:
```
def bs_call_mc(S, K, r, sigma, T, t, Ite):
    data = np.zeros((Ite, 2))
    z = np.random.normal(0, 1, [1, Ite])
    ST = S * np.exp((T - t) * (r - 0.5 * sigma**2) + sigma * np.sqrt(T - t) * z)
```

```
    data[:, 1] = ST - K

    average = np.sum(np.amax(data, axis=1)) / float(Ite)

    return np.exp(-r * (T - t)) * average
```

In [14]:
```
print(
    "European Call Price under BS (MC): ",
    bs_call_mc(100, 90, 0.05, sigma_v, 1, 0, 10000),
)
```

```
European Call Price under BS (MC):  19.447117326986664
```

Interestingly, we get prices that are very far away from each other. Is this a problem? Does our model not work?

Not at all! At the end of the day, why would the Heston option price converge to Black-Scholes?

- We have already established that BS is an oversimplification of reality (*remember: stylized facts!*)
- Heston (and other models) are just **extending the BS world** to get closer to this reality.
- The latter seems clearer when you look at Lesson 1 and the statistical distribution under Heston. There is more kurtosis and skewness than normal distribution, just as we observe in practice.
- If BS and Heston get to the same output, what is the point of adding mathematical and computational complexity?

There is another important issue related to model specification and parameters:

- In the latter pricing example, we have used a $\sigma=0.3$ for both models.
- This is not correct, since there is no easy mapping of the volatility parameter from one model to the other. In other words, the σ parameter cannot be used interchangeably in Heston or Black-Scholes.
- What is the real σ for each model then? Do not worry; we will tackle this issue through calibration in the upcoming Stochastic Modeling course.

3. Conclusion

At the beginning of the course, we introduced a simple framework to work on, i.e., the binomial model. What we have been doing since, essentially, is advancing our understanding towards more complex models that try to capture the different features associated with underlying stock prices and returns. From Black-Scholes to the Heston model, we have merely focused on modeling the underlying price, whereas the intuition on option pricing remained intact. For the final part of the Derivative Pricing course, we will focus on one more stylized fact of stock prices: jumps.

In the next lesson, we will take a look at jumps in prices, as well as how can we formally model those jumps to incorporate them in our pricing.

Copyright 2025 WorldQuant University. This content is licensed solely for personal use. Redistribution or publication of this material is strictly prohibited.

DERIVATIVE PRICING

MODULE 7 | LESSON 4

MERTON (1976) MODEL

Reading Time	120 minutes
Prior Knowledge	Volatility surface, Stochastic volatility, Heston model, Merton model: Jump diffusion
Keywords	Stochastic volatility model with stock price jumps

*In Lesson 4 of Module 7, we will implement the **Merton (1976) jump diffusion model** in Python using Monte Carlo methods. Then, we will discuss the comparison with the Black-Scholes framework and the meaning and interpretation of the different parameters.*

As usual, let's start by importing the necessary libraries:

In [1]:
```
import matplotlib.pyplot as plt
import numpy as np
import scipy.stats as ss
```

1. Implementing the Merton Model

We will start by implementing the Merton model in Python. As you already know, the model has the following SDE:

$$ \begin{equation*} dS_t = \left(r - r_j \right) S_t dt + \sigma S_t dZ_t + J_t S_t dN_t \end{equation*} $$

with the following discretized form:

$$ \begin{equation*} S_t = S_{t-1} \left(e^{\left(r-r_j-\frac{\sigma^2}{2}\right)dt + \sigma \sqrt{dt} z_t^1}+ \left(e^{\mu_j+\delta z_t^2}-1 \right) y_t \right) \end{equation*} $$

where z_t^1 and z_t^2 follow a standard normal and y_t follows a Poisson process. Finally, r_j equals to:

$$ \begin{equation*} r_j = \lambda \left(e^{\mu_j+\frac{\delta^2}{2}}\right)-1 \end{equation*} $$

Remember that, in order to obtain the parameters of the model, we will perform an exercise of calibration to option market prices. This will be, indeed, the first thing we will

cover in the following Stochastic Modeling course. For now, let's assume these parameters as given and equal to:

In [2]:
```
lamb = 0.75  # Lambda of the model
mu = 0.5  # Mu
delta = 0.25  # Delta
```

Let's also assume the following information for the current stock price, number of simulations in our Monte Carlo estimations, etc.

In [3]:
```
r = 0.05  # Risk-free rate
sigma = 0.2  # Volatility
T = 1.0  # Maturity/time period (in years)
S0 = 50  # Current Stock Price

I = 10000  # Number of simulations (paths)
M = 25  # Number of steps
dt = T / M  # Time-step
```

Next, we will calculate the random numbers that we need, together with some variable definitions for later on:

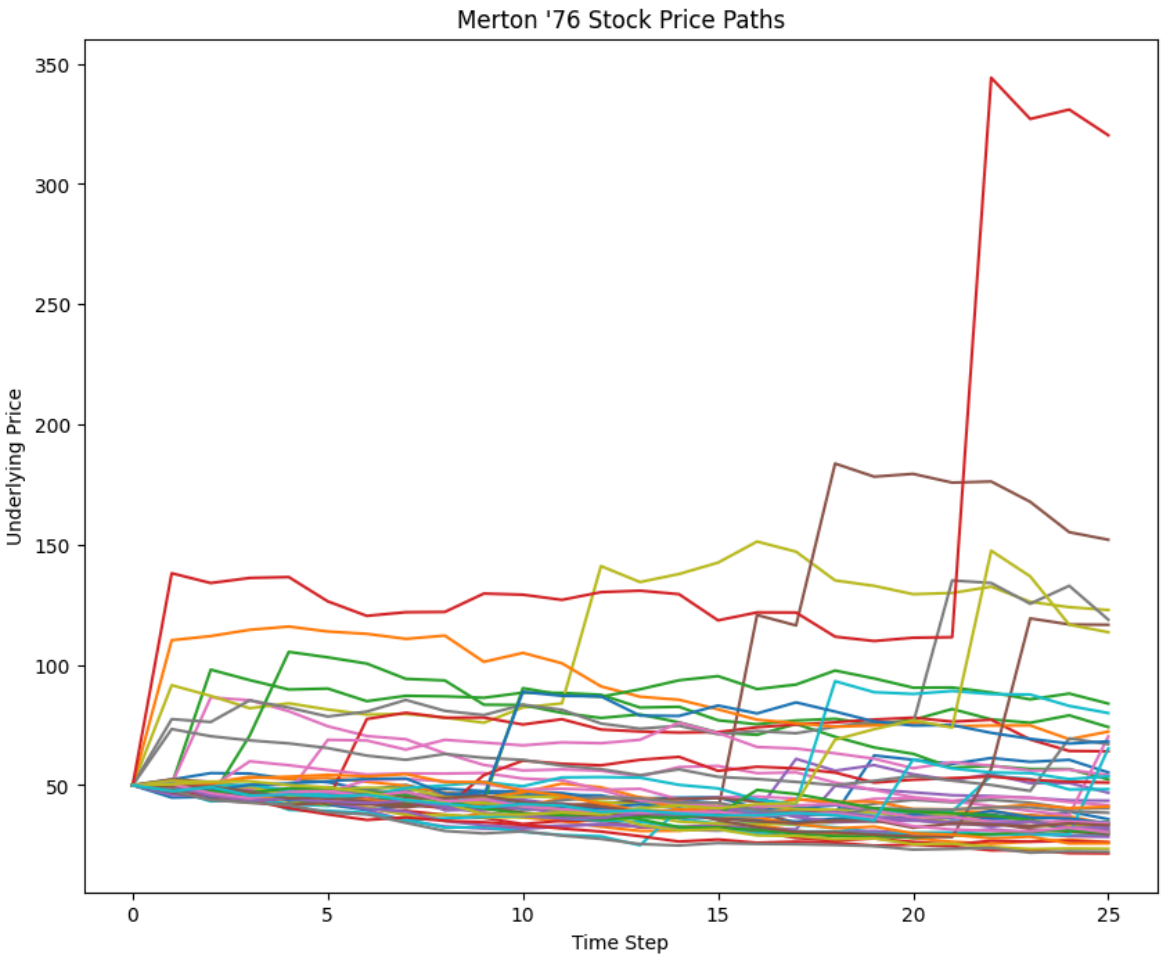
In [4]:
```
SM = np.zeros((M + 1, I))
SM[0] = S0

# rj
rj = lamb * (np.exp(mu + 0.5 * delta**2) - 1)

# Random numbers
z1 = np.random.standard_normal((M + 1, I))
z2 = np.random.standard_normal((M + 1, I))
y = np.random.poisson(lamb * dt, (M + 1, I))
```

With this info, and using the Monte Carlo methods we are familiar with already, we can simulate paths for our stock price under Merton SDE:

In [5]:
```
for t in range(1, M + 1):
    SM[t] = SM[t - 1] * (
        np.exp((r - rj - 0.5 * sigma**2) * dt + sigma * np.sqrt(dt) * z1[t])
        + (np.exp(mu + delta * z2[t]) - 1) * y[t]
    )
    SM[t] = np.maximum(
        SM[t], 0.00001
    )  # To ensure that the price never goes below zero!
```

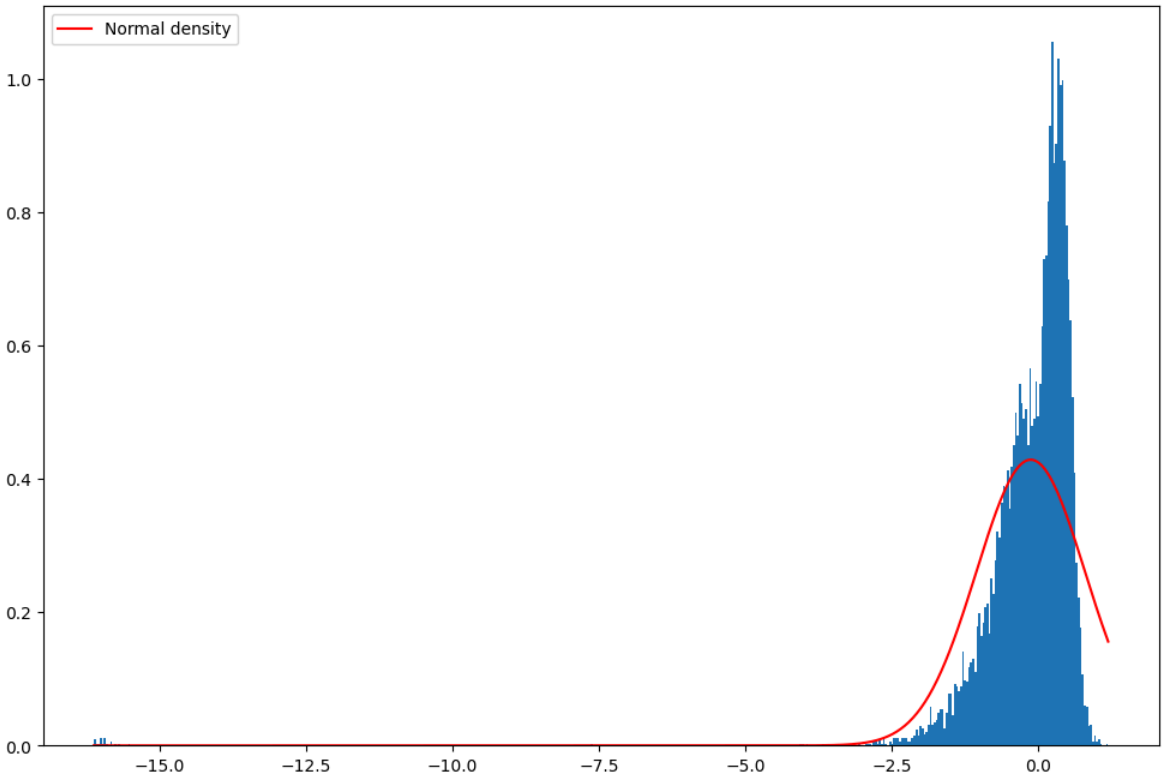
Let's see how stock price evaluation looks for a sample of 50 paths:

In [6]:
```
plt.figure(figsize=(10, 8))
plt.plot(SM[:, 100:150])
plt.title("Merton '76 Stock Price Paths")
plt.xlabel("Time Step")
plt.ylabel("Underlying Price")
plt.show()
```



What is the distribution of stock returns that this model produces?

In [9]:
```
retSM = np.log(SM[-1, :] / S0)
x = np.linspace(retSM.min(), retSM.max(), 500)

plt.figure(figsize=(12, 8))
plt.hist(retSM, density=True, bins=500)
plt.plot(
    x, ss.norm.pdf(x, retSM.mean(), retSM.std()), color="r", label="Normal densi
)
plt.legend()
plt.show()
```



As you can see, the distribution generated by the Merton model has more kurtosis than a normal distribution and a negative skewness, consistent with the different stylized facts of stock returns. We will, later on, discuss how changing the different parameters modifies this distribution. First, let's compare this result to the returns produced by GBM and Heston.

2. Comparison to Heston and GBM Returns

Next, we will compare the return (and its distribution) generated by other models like Black-Scholes (GBM) or Heston.

Let's start with the former and, when possible, use the same parameters as in the previous example.

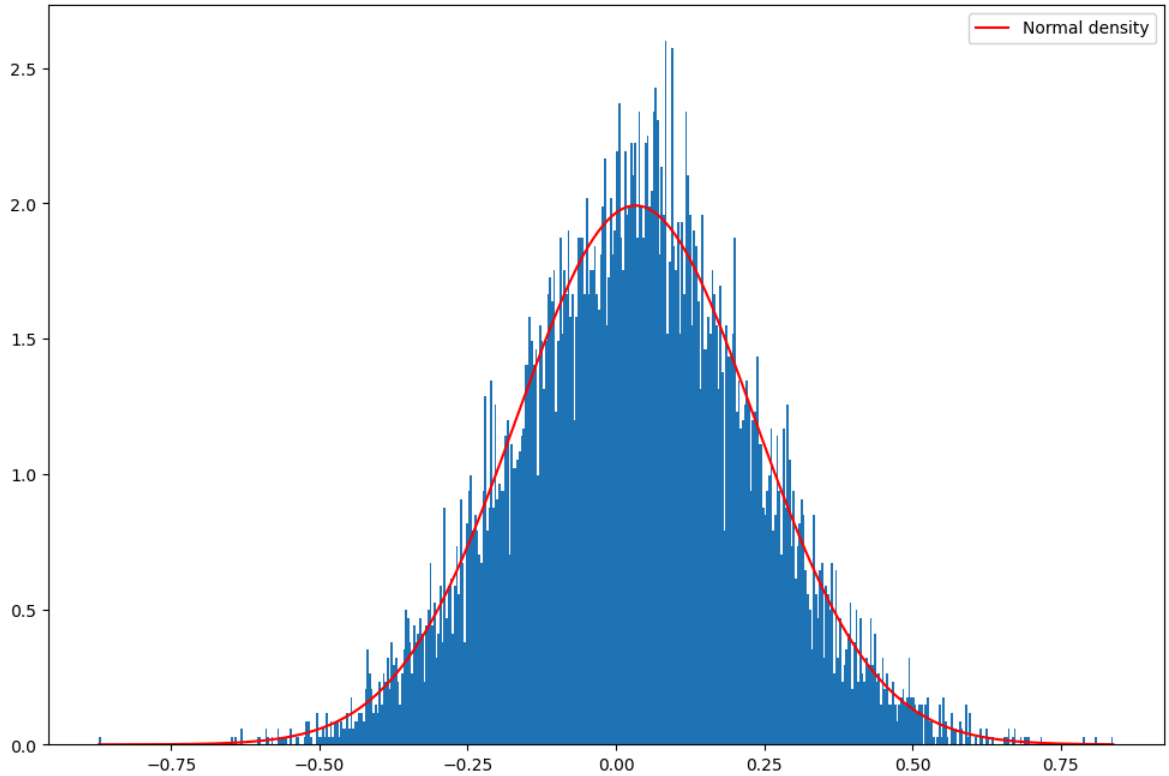
2.1. GBM Returns and Distribution

Let's implement the classic GBM and check the distribution of its returns:

In [10]:
```
S = np.zeros((M + 1, I))
S[0] = S0
for t in range(1, M + 1):
    S[t] = S[t - 1] * np.exp(
        (r - 0.5 * sigma**2) * dt
        + sigma * np.sqrt(dt) * np.random.standard_normal(I)
    )
```

In [11]:
```
retS = np.log(S[-1, :] / S0)
y = np.linspace(retS.min(), retS.max(), 500)
```

In [12]:
```
plt.figure(figsize=(12, 8))
plt.hist(retS, density=True, bins=500)
plt.plot(y, ss.norm.pdf(y, retS.mean(), retS.std()), color="r", label="Normal de
plt.legend()
plt.show()
```



Obviously, what we get here is the classic normal distribution.

2.2. Heston Returns and Their Distribution

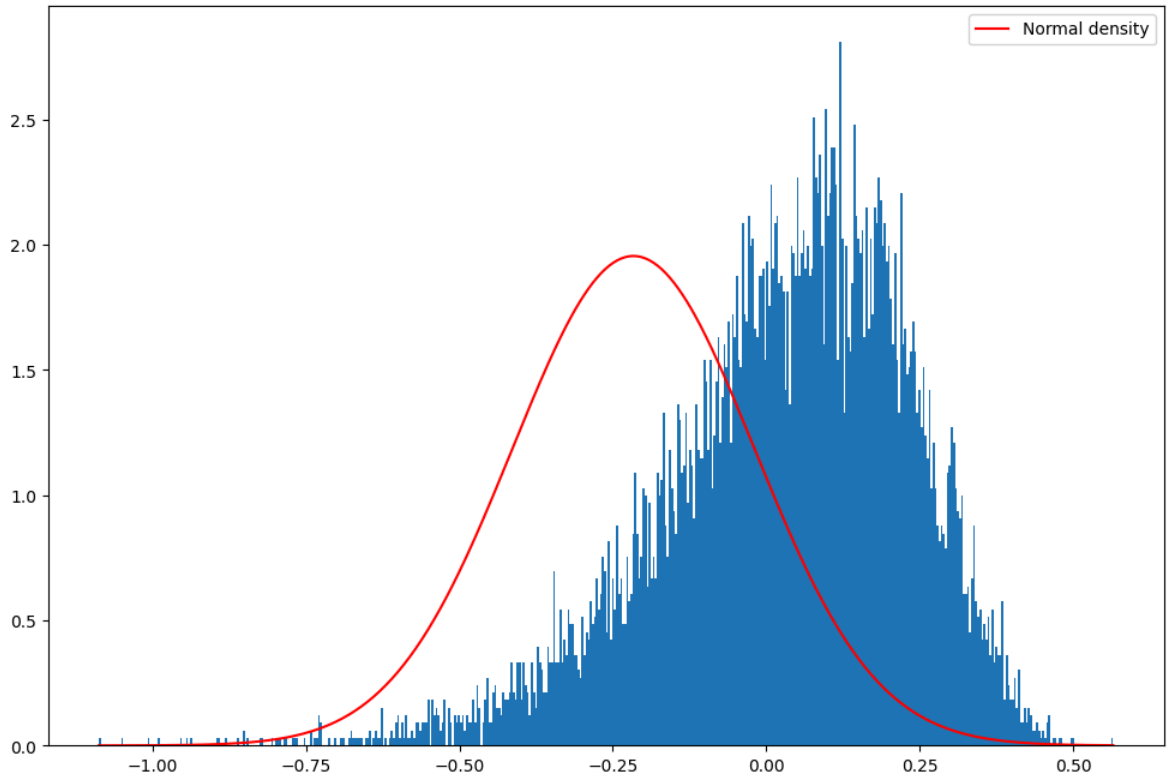
Finally, let's compare this to Heston-produced returns:

In [13]:
```
def SDE_vol(v0, kappa, theta, sigma, T, M, I, rand, row, cho_matrix):
    dt = T / M  # T = maturity, M = number of time steps
    v = np.zeros((M + 1, I), dtype=float)
    v[0] = v0
    sdt = np.sqrt(dt)  # Sqrt of dt
    for t in range(1, M + 1):
        ran = np.dot(cho_matrix, rand[:, t])
        v[t] = np.maximum(0, v[t - 1] + kappa * (theta - v[t - 1]) * dt + np.sqr
    return v

def Heston_paths(S0, r, v, row, cho_matrix):
    S = np.zeros((M+1, I), dtype=float)
    S[0] = S0
    sdt = np.sqrt(dt)
    for t in range(1, M+1, 1):
        ran = np.dot(cho_matrix, rand[:,t])
        S[t] = S[t - 1] * np.exp((r - 0.5 * v[t-1]) * dt + np.sqrt(v[t-1]) * ran

    return S

def random_number_gen(M, I):
```

```
    rand = np.random.standard_normal((2, M+1, I))
    return rand
```

In [14]:
```
# Heston given parameters
v0 = 0.04
kappa_v = 2
sigma_v = 0.2
theta_v = 0.04
rho = -0.9


# Generating random numbers from standard normal
rand = random_number_gen(M, I)


# Covariance Matrix
covariance_matrix = np.zeros((2, 2))
covariance_matrix[0] = [1.0, rho]
covariance_matrix[1] = [rho, 1.0]
cho_matrix = np.linalg.cholesky(covariance_matrix)

# Volatility process paths
V = SDE_vol(v0, kappa_v, theta_v, sigma_v, T, M, I, rand, 1, cho_matrix)

# Underlying price process paths
HS = Heston_paths(S0, r, V, 0, cho_matrix)
```

In [15]:
```
retHS = np.log(HS[-1, :] / S0)
q = np.linspace(retHS.min(), retHS.max(), 500)


plt.figure(figsize=(12, 8))
plt.hist(retHS, density=True, bins=500)
plt.plot(
    q, ss.norm.pdf(y, retHS.mean(), retHS.std()), color="r", label="Normal densi
)
plt.legend()
plt.show()
```

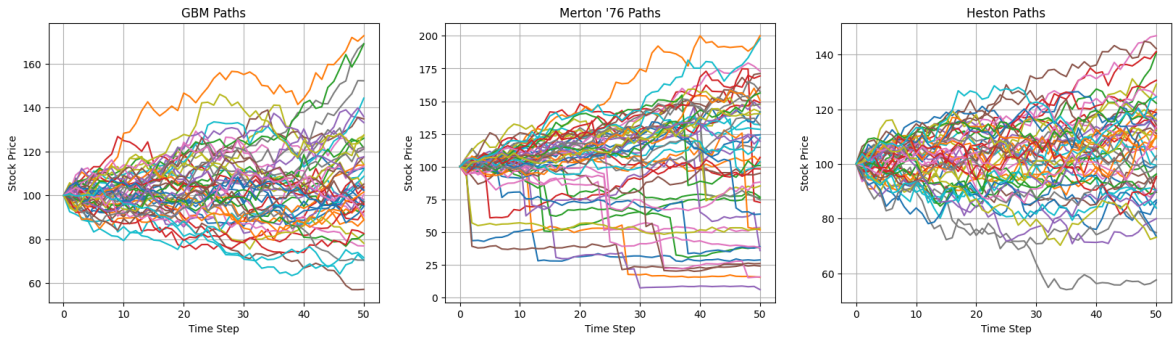


As we have seen in the previous lessons, this is the distribution of returns produced by Heston.

These clear differences between the different models are also evident just by looking at the kind of stock price paths they produce:

In [16]:
```
fig = plt.figure(figsize=(20, 5))
ax1 = fig.add_subplot(131)
ax2 = fig.add_subplot(132)
ax3 = fig.add_subplot(133)

ax1.plot(S[:, :50])
ax1.grid()
ax1.set_title("GBM Paths")
ax1.set_ylabel("Stock Price")
ax1.set_xlabel("Time Step")

ax2.plot(SM[:, :50])
ax2.grid()
ax2.set_title("Merton '76 Paths")
ax2.set_ylabel("Stock Price")
ax2.set_xlabel("Time Step")

ax3.plot(HS[:, :50])
ax3.grid()
ax3.set_title("Heston Paths")
ax3.set_ylabel("Stock Price")
ax3.set_xlabel("Time Step")
```

Out[16]:
```
Text(0.5, 0, 'Time Step')
```



3. Option Pricing under the Different Models

Next, let's explore the consequences of these differences in option pricing. For simplicity, let's assume a European call option with $K=90$ and $T=1$ year. Let's use Monte Carlo methods for this and assume the parameters from before.

3.1. Pricing under Black-Scholes (GBM)

In [17]:
```
def bs_call_mc(S, K, r, sigma, T, t, I):
    data = np.zeros((I, 2))
    z = np.random.normal(0, 1, [1, I])
    ST = S * np.exp((T - t) * (r - 0.5 * sigma**2) + sigma * np.sqrt(T - t) * z)
    data[:, 1] = ST - K

    average = np.sum(np.amax(data, axis=1)) / float(I)

    return np.exp(-r * (T - t)) * average
```

In [18]:
```
print("European Call Price under BS (MC): ", bs_call_mc(S0, 90, r, sigma, T, 0,
```

```
European Call Price under BS (MC):  16.76783971774069
```

3.2 Pricing under Heston

In [19]:
```
def heston_call_mc(S, K, r, T, t):
    payoff = np.maximum(0, S[-1, :] - K)

    average = np.mean(payoff)

    return np.exp(-r * (T - t)) * average
```

In [20]:
```
print("European Call Price under Heston: ", heston_call_mc(HS, 90, r, T, 0))
```

```
European Call Price under Heston:  16.959109347421297
```

3.3 Pricing under Merton

In [21]:
```
def merton_call_mc(S, K, r, T, t):
    payoff = np.maximum(0, S[-1, :] - K)

    average = np.mean(payoff)

    return np.exp(-r * (T - t)) * average
```

In [22]:
```
print("European Call Price under Merton: ", merton_call_mc(SM, 90, r, T, 0))
```

```
European Call Price under Merton:  27.70407381672255
```

Are these results for the price of the options consistent with the distributions we observed?

4. Discussion of Merton Model Parameters

Finally, let's discuss empirically how the different parameters from Merton influence the type and form of the distribution of stock returns. Specifically, we will focus on the parameters λ and μ_j:

- λ → Intensity (frequency) of the jump (shock) to stock prices
- μ_j → Average jump size (can be positive or negative)

As we have already mentioned more than a few times, the actual values that we will use for these parameters will come from a calibration to observed option market prices.

While we will tackle this issue soon, for now, it will be helpful to understand what the sign and magnitude of these parameters mean from an empirical standpoint.

4.1. Changing λ

In the previous example, we used a $\lambda=0.75$, which accounted for the intensity (frequency) of the jumps in interval t. In this case, we expect 0.75 events in a year on average. In other words, the average rate at which jumps occur is 0.75 (75%).

What will happen to the previous distribution of returns if we increase (or decrease) this number?

Let's check it by imposing $\lambda=0.99$:

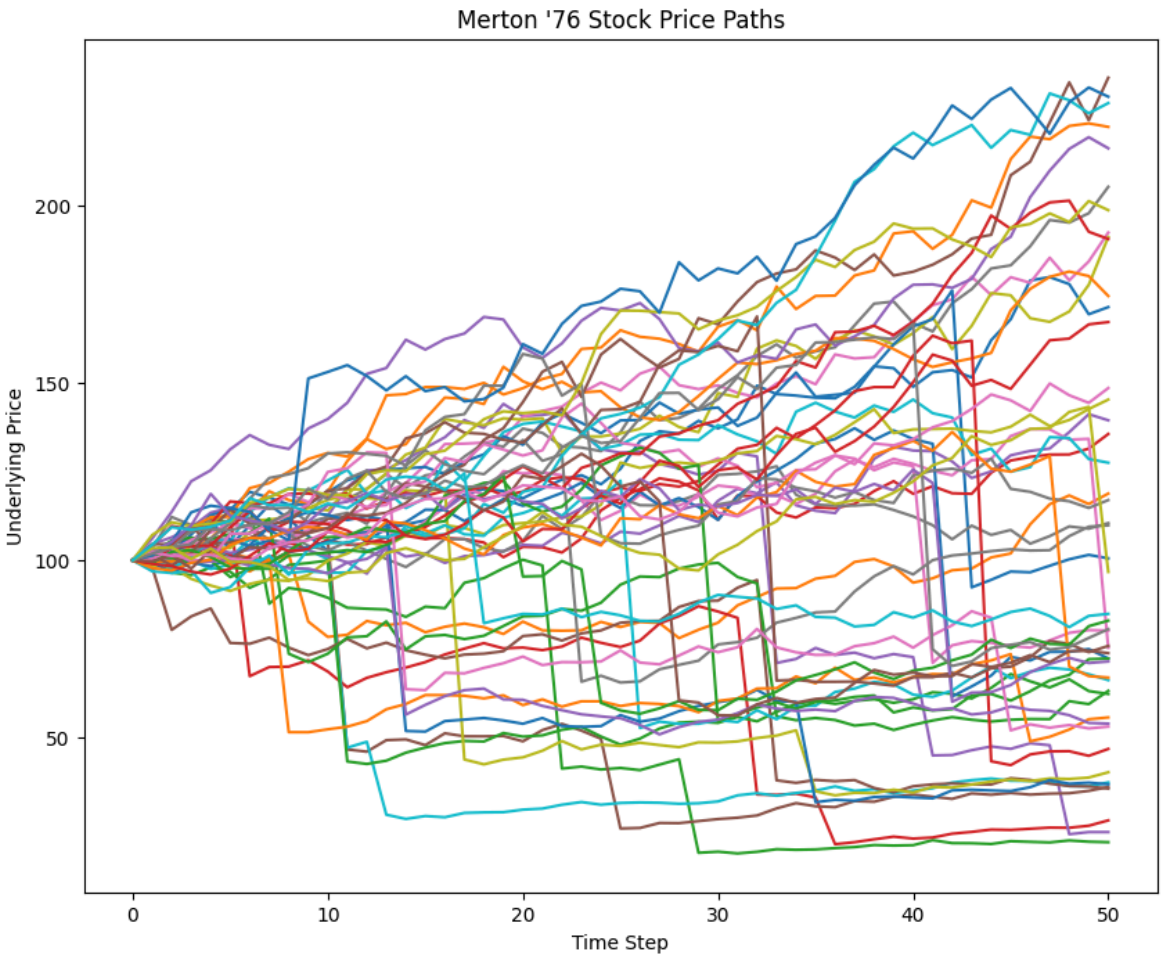
In [23]:
```
lamb = 0.75  # Lambda of the model
mu = 0.6  # Mu
delta = 0.25  # Delta

r = 0.05  # Risk-free rate
sigma = 0.2  # Volatility
T = 1.0  # Maturity/time period (in years)
S0 = 100  # Current Stock Price

I = 10000  # Number of simulations (paths)
M = 50  # Number of steps
dt = T / M  # Time-step

SM = np.zeros((M + 1, I))
SM[0] = S0

# rj
rj = lamb * (np.exp(mu + 0.5 * delta**2) - 1)
```

```
# Random numbers
z1 = np.random.standard_normal((M + 1, I))
z2 = np.random.standard_normal((M + 1, I))
y = np.random.poisson(lamb * dt, (M + 1, I))
```

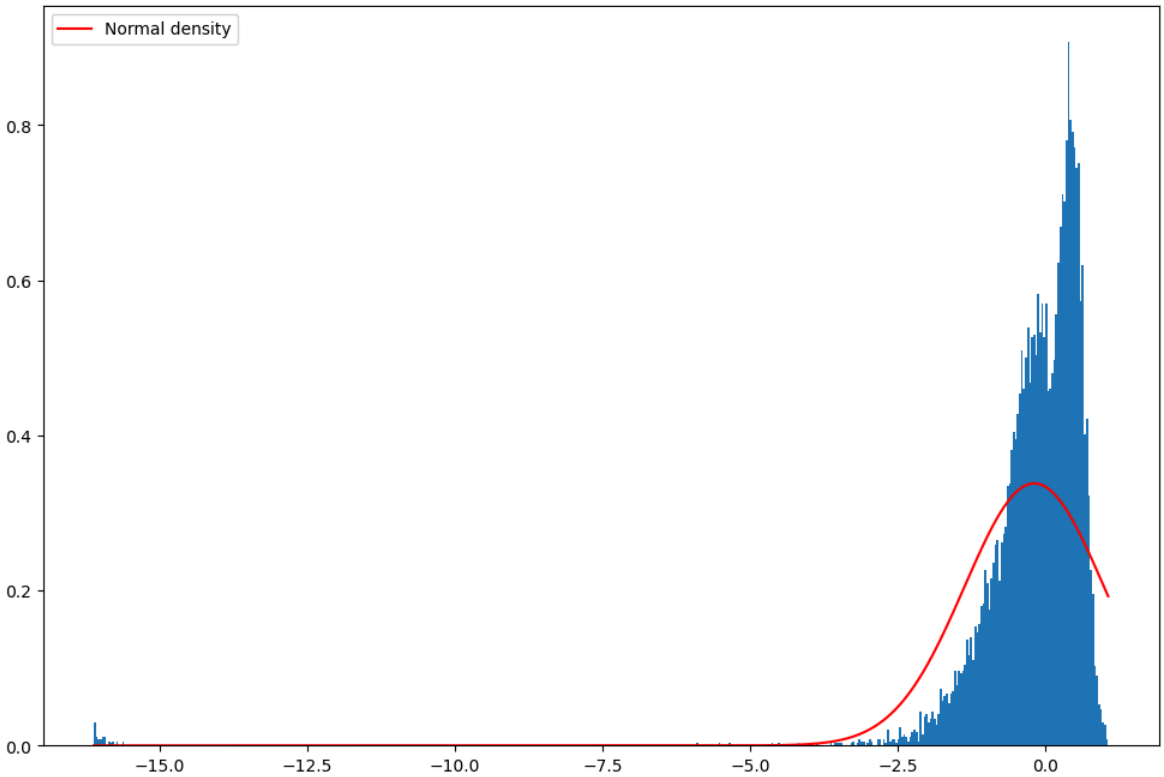
In [24]:
```
lamb = 0.99  # Lambda of the model
mu = -0.6  # Mu
delta = 0.25  # Delta

r = 0.05  # Risk-free rate
sigma = 0.2  # Volatility
T = 1.0  # Maturity/time period (in years)
S0 = 100  # Current Stock Price

I = 10000  # Number of simulations (paths)
M = 50  # Number of steps
dt = T / M  # Time-step

SM = np.zeros((M + 1, I))
SM[0] = S0

# rj
rj = lamb * (np.exp(mu + 0.5 * delta**2) - 1)

# Random numbers
z1 = np.random.standard_normal((M + 1, I))
z2 = np.random.standard_normal((M + 1, I))
y = np.random.poisson(lamb * dt, (M + 1, I))
```

In [25]:
```
for t in range(1, M + 1):
    SM[t] = SM[t - 1] * (
        np.exp((r - rj - 0.5 * sigma**2) * dt + sigma * np.sqrt(dt) * z1[t])
        + (np.exp(mu + delta * z2[t]) - 1) * y[t]
    )
    SM[t] = np.maximum(
        SM[t], 0.00001
    )  # To ensure that the price never goes below zero!
```

In [26]:
```
plt.figure(figsize=(10, 8))
plt.plot(SM[:, 100:150])
plt.title("Merton '76 Stock Price Paths")
plt.xlabel("Time Step")
plt.ylabel("Underlying Price")
plt.show()
```



In [27]:
```
retSM = np.log(SM[-1, :] / S0)
x = np.linspace(retSM.min(), retSM.max(), 500)

plt.figure(figsize=(12, 8))
plt.hist(retSM, density=True, bins=500)
plt.plot(
    x, ss.norm.pdf(x, retSM.mean(), retSM.std()), color="r", label="Normal densi
)
plt.legend()
plt.show()
```



If you compare this density graph to the one we obtained in the case of $\lambda=0.75$, you will observe that the main difference occurs in the extreme left tail of the distribution. This makes sense, since by increasing λ we are effectively influencing the probability that the stock price experiences more negative returns (jumps).

What if we decrease λ?

4.2. Changing μ_j

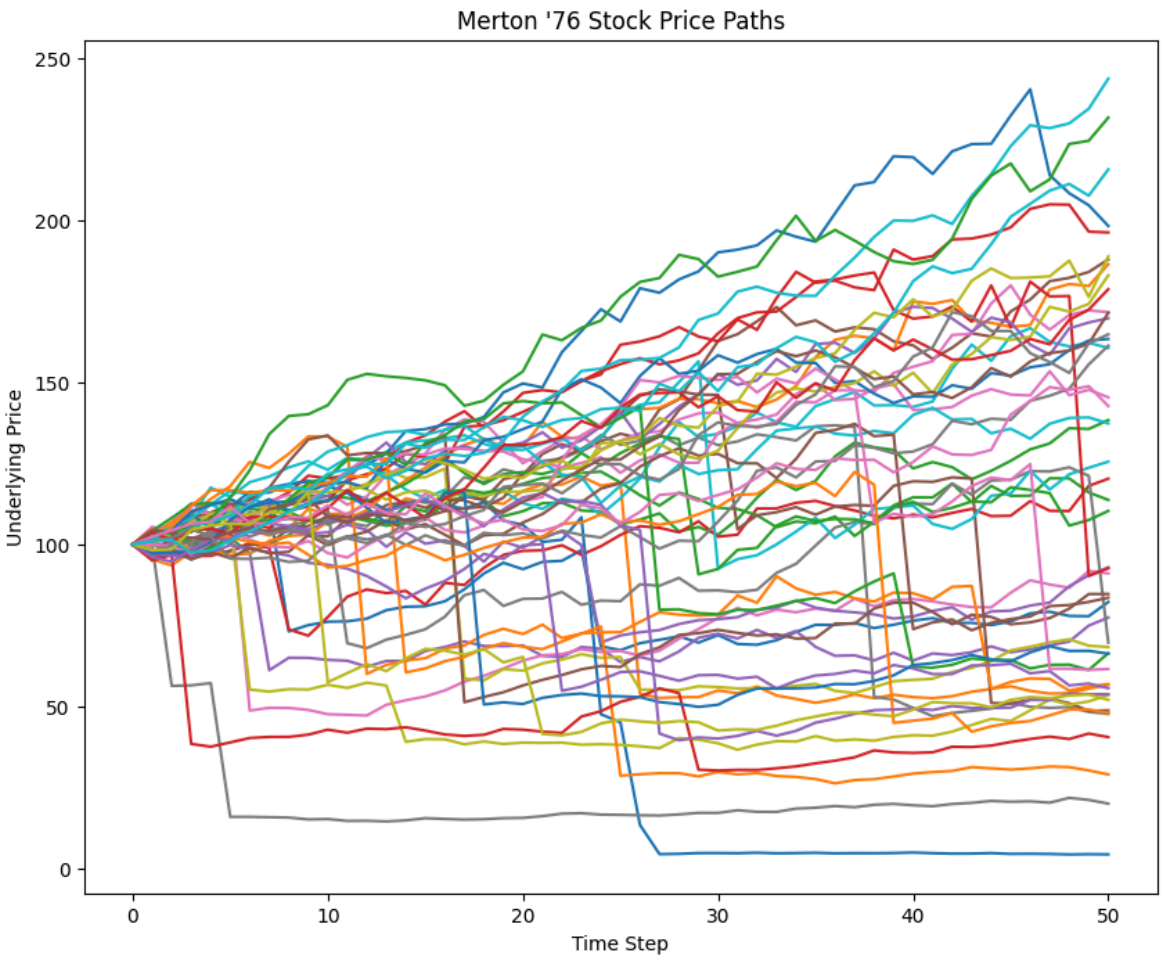
In the previous case, we observe how the distribution's negative skewness increases as a consequence of a higher $λ$. But why the left tail? Essentially, because we have a $\mu_j<0$. That is, the average jump size is negative. This means that every time the stock price experiences a jump, there is a very high likelihood that this is a negative jump.

What if we set $\mu_j>0$?

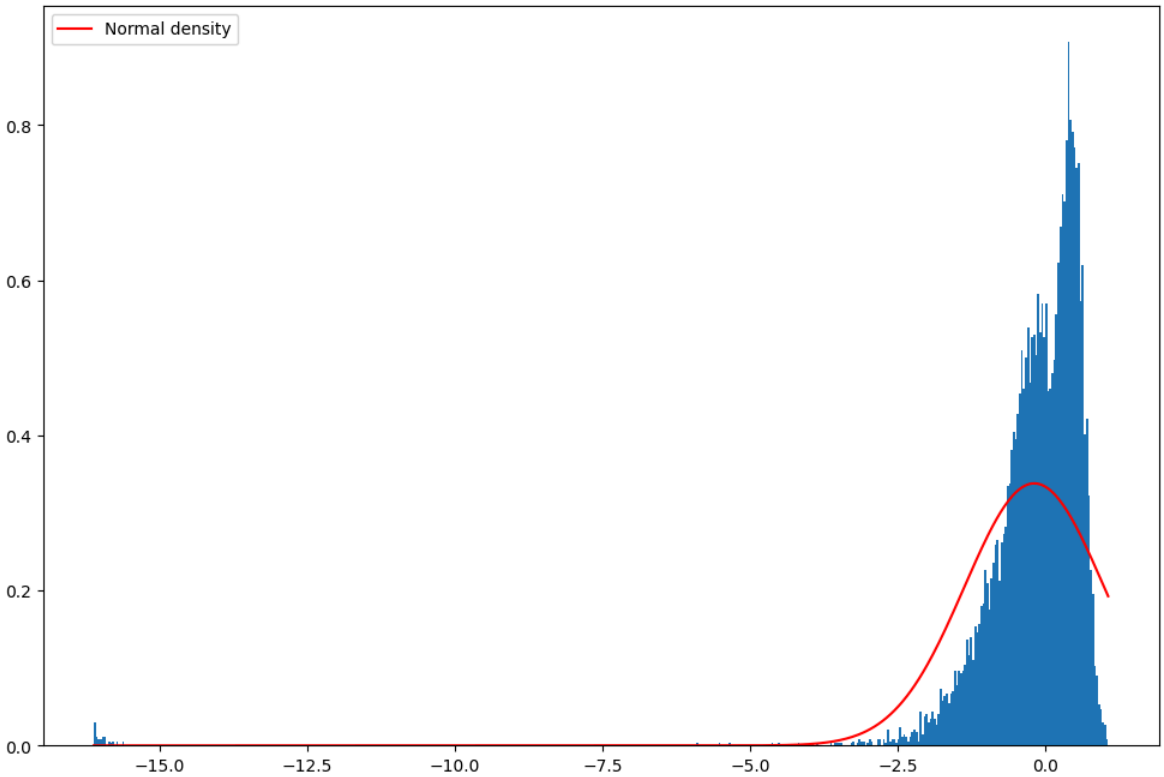
In [28]:
```
for t in range(1, M + 1):
    SM[t] = SM[t - 1] * (
        np.exp((r - rj - 0.5 * sigma**2) * dt + sigma * np.sqrt(dt) * z1[t])
        + (np.exp(mu + delta * z2[t]) - 1) * y[t]
    )
    SM[t] = np.maximum(
        SM[t], 0.00001
    )  # To ensure that the price never goes below zero!
```

In [29]:
```
plt.figure(figsize=(10, 8))
plt.plot(SM[:, 0:50])
plt.title("Merton '76 Stock Price Paths")
plt.xlabel("Time Step")
```

```
plt.ylabel("Underlying Price")
plt.show()
```



In [30]:
```
retSM = np.log(SM[-1, :] / S0)
x = np.linspace(retSM.min(), retSM.max(), 500)

plt.figure(figsize=(12, 8))
plt.hist(retSM, density=True, bins=500)
plt.plot(
    x, ss.norm.pdf(x, retSM.mean(), retSM.std()), color="r", label="Normal densi
)
plt.legend()
plt.show()
```



Now, as you see, the distribution of returns has skewness (heavier right tail) than normal.

4.3. Changing δ_j

What will happen if we change δ_j? See for yourself!

5. Conclusion

Nicely done! That is all for the Derivative Pricing course. We are not done with these models, however, as we will revisit both of the models covered in this module in the next Stochastic Modeling course. We will not only advance our understanding of model calibration but also look at how we can provide semi-closed form solutions to the pricing problem embedded in Heston and Merton's models!

See you there!

Copyright 2025 WorldQuant University. This content is licensed solely for personal use. Redistribution or publication of this material is strictly prohibited.

LESSON 1

STOCHASTIC VOLATILITY: HESTON

Derivative Pricing - Module 7

WORLDQUANT UNIVERSITY

Outline

- Intro to Stochastic Volatility
- The Heston model
- Monte-Carlo methods for Heston

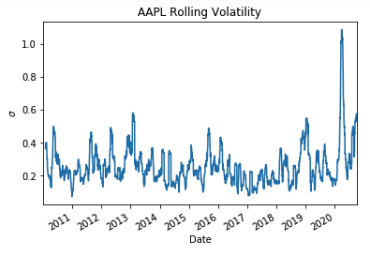
Stochastic Volatility

→ In past modules, we extended Black-Scholes considering models that allowed for some elasticity of variance (**local volatility**).

→ While this helped towards replicating the **volatility smile**, it is not enough.

→ In this module, we will introduce **stochastic volatility** models, specifically Heston (1993).

- Heston incorporates **2 important features**: Volatility is **(i) stochastic** and **(ii) mean-reverting**.



Heston Model

The Heston model comprises, essentially, two SDEs: one for the asset (Eq. 1) and one for its variance (Eq. 2):

$$dS = \mu S dt + \sqrt{\nu(t)} S dZ_1 \tag{1}$$

$$d\nu(t) = \kappa\left(\theta - \nu(t)\right) dt + \sigma\sqrt{\nu(t)} dZ_2 \tag{2}$$

where dZ_1 and dZ_2 are Wiener processes with correlation ρ.

- In Equation 1 (**SDE for asset**), under a risk-neutral setting, we'll have $\mu = r$.
- Equation 2 (**SDE for variance**) is what is new to us (is it though? $\rightarrow$ remember Vasicek?)
  - θ, long run average variance.
  - κ, rate of mean-reversion.
  - σ, volatility of volatility (vol of vol).

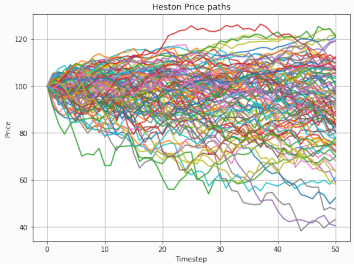
Simulating Heston

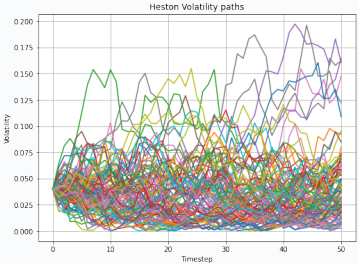
As before, given some parameters, we can **simulate Heston under Monte-Carlo** methods:

$$S_t = S_{t-1} e^{\left(r - \frac{\nu_t}{2}\right) dt + \sigma \sqrt{\nu_t} dZ_1}$$

$$\nu_t = \nu_{t-1} + \kappa \left(\theta - \nu_{t-1}\right) dt + \sigma \sqrt{\nu_{t-1}} dZ_2$$

One important input here will be the different parameters. For now, let's assume some and use MC to simulate:





Summary of Lesson 1

In Lesson 1, we have looked at:

- Stochastic volatility
- The Heston (1993) stochastic volatility model

⇒ **TO DO NEXT**: Now, please go to the associated Jupyter notebook for this lesson for a full development in Python of the Heston model under Monte-Carlo methods.

⇒ In the next lesson, we will take a closer look at option pricing in the Heston framework.

WORLDQUANT UNIVERSITY

LESSON 2

OPTION PRICING UNDER HESTON (MONTE-CARLO)

Derivative Pricing - Module 7

WORLDQUANT UNIVERSITY

Outline

- Using the Heston model: Problems at hand
- Option pricing in the Heston framework
- Heston vs. Black-Scholes pricing

Using Heston

We now know how to use Monte Carlo (MC) to simulate Heston dynamics. However, there is still an unsolved question:

$\Rightarrow$ **How do we obtain Heston model parameters (κ, θ, σ, ...) ?**

1. One option is to **calibrate the model** to market prices and volatility (similar to what we did with CEV).

   However, you must realize at this point that calibrating using MC on Heston original SDEs entails **huge computational costs**.

2. Another option is to work on a **semi-analytical solution** (methods to approximate a numerical solution).

   This consists of the use of methods to approximate a numerical solution. Semi-analytical solutions will **save a lot of time** when it comes to model calibration and hedging, an important aspect to consider.

We will specifically tackle these problems in the **Stochastic Modeling** course.

For now, we will learn, in a lab-like setting, how to **price options under Heston** using Monte-Carlo methods.

Option Pricing in Heston

At this point, you should have already realized that the **hard part** of option pricing is in the **underlying asset price dynamics**:

- Once we have assumed some **(risk-neutral) behavior for the underlying**, the price of the option is merely its **(discounted) payoff.**
  - If we are lucky, there may exist a **closed-form solution** to option pricing (e.g., Black-Scholes).
  - There is always the option to use **Monte-Carlo methods** to simulate the expected option payoff.

The Heston model is no exception. We will learn about its closed-form solution in the next course, but we are already prepared to price options under these dynamics using Monte-Carlo.

Thus, the price of a European Call Option will be:

$$C_0 = e^{-rT} E\left(S_T - K\right)^+$$

where S_T follows the (risk-neutral) underlying price dynamics of Heston model.

Heston vs. Black-Scholes Pricing

In the Jupyter notebook accompanying this lesson, you will find an example of option pricing under the Heston model, using Monte-Carlo methods. Then, we compare this price to what we would have obtained through Black-Scholes.

$\Rightarrow$ **Would these two prices converge?** No!

- Heston extends the BS framework to **incorporate more features** that we observe in practice.
  - Heston produces returns with more kurtosis & skewness, as in practice!
- What's the **point of adding mathematical & computational complexity** if we arrive at the same price?
- Are the parameters (e.g., σ) the **same for both models**? No.
  - In BS, σ is stock volatility; in Heston, σ is vol of vol.
- How can we know the **parameters of each model**? $\rightarrow$ Calibration
  - Remember CEV? We will obtain model parameters by calibration to observed market prices.

Summary of Lesson 2

In Lesson 2, we have looked at:

- The problems that arise in implementing the Heston model
- How to price options under Heston model dynamics using Monte Carlo
- Heston vs. Black-Scholes option pricing

⇒ **TO DO NEXT**: Now, please go to the associated Jupyter notebook for this lesson for a Python example of option pricing under the Heston model using Monte-Carlo methods.

⇒ In the next lesson, we will take a look at another important characteristic of stock returns: jumps.

LESSON 3

JUMP DIFFUSION MODELS

Derivative Pricing - Module 7

WORLDQUANT UNIVERSITY

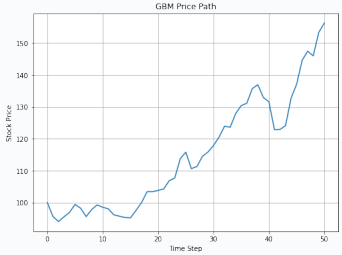
Outline

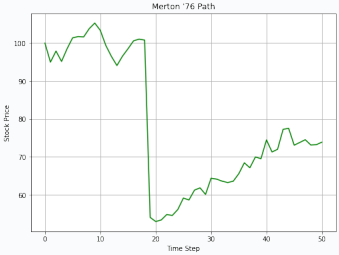
- Jump diffusion models
- Merton (1976) model
- Poisson processes

Jump diffusion models

We started with a **normally distributed world** (e.g., Black-Scholes), and moved to one in which **volatility is non-constant** (e.g., CEV) and **stochastic** (e.g., Heston). Our focus has been primarily on the volatility part of the underlying price process. Let's now turn to the **drift component**.

- The GBM model (even in Heston) produces an arguably "smooth" stock price evolution.
- However, this is not really what happens in reality, where prices constantly experience jumps, e.g.,:
  - Due to good (e.g., earnings surprise) or bad (e.g., profit warning) **fundamental reasons**.





Merton (1976) model

One of the most famous jump diffusion models for option pricing is **Merton (1976)**:

Merton, Robert C. "Option pricing when underlying stock returns are discontinuous." Journal of Financial Economics, vol. 3, no. 1–2, 1976, pp. 125–144.

Merton's '76 model SDE includes both a Brownian motion and a Poisson jump process with intensity λ and average jump size μ_j:

$$dS_t = \left(r - r_j\right) S_t dt + \sigma S_t dZ_t + J_t S_t dN_t$$

where,

- $r_j = \lambda \left(e^{\mu_j + \frac{\delta^2}{2}} \right) - 1$

  is the "correction" to the drift term in order to maintain the risk-neutral measure.

- J_t is a jump at time t, which follows the distribution:

$$Log(1 + J_t) \approx Normal \left(log(1 + \mu_j) - \frac{\delta^2}{2},\ \delta^2 \right)$$

Merton (1976) model

Similar to what we have done with other previous models, we can use Monte Carlo methods on the discretized version of the model for pricing:

$$S_t = S_{t-1}\left(e^{\left(r - r_j - \frac{\sigma^2}{2}\right)dt + \sigma\sqrt{dt}z_t^1} + \left(e^{\mu_j + \delta z_t^2} - 1\right) y_t\right)$$

Note that we have **3 sources of randomness** here:

1. Stock price diffusion (standard normal): z_t^1
2. Size of the jump (standard normal): z_t^2
3. Timing of the jump (Poisson): y_t

Also, note the importance of variables μ_j and λ, the average size and intensity (frequency) of the jump:

- How to assess these parameters? $\rightarrow$ Calibration
- In the next lesson, we will implement the model and play around with these.

Poisson processes

Merton (1976) models stock price jumps in a very convenient way using random processes following a **Poisson distribution**.

If you are familiar with Poisson distributions, this should not surprise you, as it is a process often used to model the **occurrence of events**. Some applications are:

- Traffic flow $\rightarrow$ theory of queues, how long should I wait for the bus?
- Firm bankruptcy $\rightarrow$ assess the probability of default.
- Insurance claims $\rightarrow$ probability of reporting an accident & assessment of insurance premium.

The probability mass function of a Poisson-distributed random variable X is:

$$P(X = k) = \frac{\lambda^k e^{-\lambda}}{k!}$$

where k is the number of occurrences of the event, and $\lambda > 0$.

$\Rightarrow$ Also, note that $\lambda = E(X) = Var(X)$

Summary of Lesson 3

In Lesson 3, we have looked at:

- Jump diffusion models as a way to get closer to real-life behavior of prices.
- Merton (1976) jump diffusion model.
- Poisson process basics and its applications to finance.

⇒ **TO DO NEXT**: This lesson does not have an associated Jupyter Notebook. Jump to the next lesson to see a complete implementation of the Merton (1976) model in Python, together with a discussion of the distribution that it generates for stock prices/returns and the parameters of the model.

⇒ In the next lesson, you will learn how to implement the Merton (1976) model in Python.